

IDS Fundamentals

Contents

- IDS Fundamentals..... 1
 - Introduction 2
 - Types of IDS..... 2
 - Deployment Modes: 2
 - Detection Modes: 3
- IDS Example: Snort 3
 - SNORT USAGE 5
 - Rule format 5
 - Rule creation..... 6
 - Rule testing..... 6
 - Running Snort on PCAP files 7
- PRACTICAL..... 8

Introduction

Firewalls are the security solution usually deployed on the boundary of a network to protect its incoming and outgoing traffic. The firewall checks the traffic when a connection is going to take place and denies it if it violates the firewall rules.

Firewall = a security tool, hardware or software that is used to filter network traffic by stopping unauthorized incoming and outgoing traffic.

There should also be some security to detect the activities of the connections that passed through the firewall and has already taken place. So, if an attacker successfully bypasses a firewall via a legitimate-looking connection and then performs any malicious activities inside the network, there should be something to detect it in a timely manner. For this purpose, we have a security solution inside the network. This solution is known as an Intrusion Detection System (IDS).

Firewall basically acts as the gatekeeper, checking the people coming in and going out. There is always a chance that some bad actor will successfully sneak inside and start performing malicious activities. He was missed at the gate, but what if we catch him even after he gets in? This can be done by the surveillance cameras present throughout the building. The IDS plays the role of surveillance cameras. It monitors the network traffic based on its signature and anomaly-based detections, and detects abnormal traffic going out or inside the network. > after a detection alert is generated for the security admin. IDS DOESN'T ACT ON THOSE DETECTION!!

Types of IDS

Deployment Modes:

- Host Intrusion Detection System (HIDS): Host-based IDS solutions are installed individually on the hosts and are responsible for only detecting potential security threats associated with the particular host. They provide detailed visibility of the host's activities. However, host intrusion detection systems can be challenging to manage in large networks as they are resource-intensive and require management on each host.
- Network Intrusion Detection System (NIDS): Network-based IDS solutions are crucial in detecting potentially malicious activities within the whole network, regardless of any specific hosts. They monitor the network traffic of all the hosts involved to detect suspicious activities. It provides a centralized view of all the detections inside the whole network.

Detection Modes:

- Signature-based IDS: Many attacks occur every day. Each attack has its unique pattern, which is known as a signature. These signatures are preserved by the IDS in their databases so that if the same attack happens in the future, it gets detected by its signature and reported to the security admins for action.
ATTENTION! > Signature-based IDS is unable to detect zero-day attacks. Zero-day attacks have no prior signatures (patterns) and are not saved inside the IDS databases. Signature-based IDS can detect only attacks that have previously happened and have a signature(pattern) saved inside the database. **Example: Snort**
- Anomaly-based IDS: This type of IDS first learns the normal behaviour (baseline) of the network or system and performs detections if there is any deviation from the normal behavior. Anomaly-based IDS can also detect zero-day attacks because they don't rely on available signatures for detection. They detect abnormalities inside the network or system by comparing the current state with the baseline. However, this type of IDS may generate a lot of false positives (marking benign activities as malicious), because the nature of most legitimate programs matches the malicious ones. For the best usage of anomaly-based IDS we need fine-tuning it (manually defining the normal behaviour in the IDS).
- Hybrid IDS: Combination of Signature and Anomaly-based IDS.

IDS Example: Snort

Snort is one of the most widely used open-source IDS solutions developed in 1998. It uses signature-based and anomaly-based detections to identify known threats. These are defined in the rule files of the Snort tool. Several built-in rule files come preinstalled in this tool's package. The basic built-in rules can detect a lot of malicious traffic for us, but we can also configure our own detection rules.

Usage of Snort

Mode	Description	Use Case
Packet sniffer mode	This mode reads and displays network packets without performing any analysis on them. The packet sniffer mode of Snort does not directly relate to IDS capabilities, but it can be helpful in network monitoring and troubleshooting.	The network team observes some network performance issues. To diagnose the issue, they need detailed insights into the network traffic.

Packet logging mode	Snort performs detection on the network traffic in real-time and displays the detections as alerts on the console for the security admins to take action. For later analysis the network traffic needs to be logged for later analysis > PCAP file that contains network traffic and any detections from it. > helpful to forensic investigators	The security team needs to initiate a forensic investigation of a network attack. They would need the traffic logs to perform the root cause analysis.
NIDS mode	Snort's NIDS mode is the primary mode that monitors network traffic in real-time and applies its rule files to identify any match to the known attack patterns stored as signatures. If there is a match, it generates an alert	The security team must proactively monitor their network or systems to detect potential threats.

SNORT USAGE

During Snort installation, you must provide your network interface and range. You can run Snort normally, where it only captures the traffic intended for your host. However, if you want to use Snort to capture and detect intrusions in your whole network, you must turn on the promiscuous (wide-range) mode of your host's network interface.

Snort built-in rule files and configuration file etc... are stored in the /etc/snort directory. In Snort 3, this location is not fixed, it is chosen during an installation.

```
ubuntu@tryhackme:~$ ls /etc/snort
Intro_to_IDS.pcap      gen-msg.map           snort.conf            threshold.conf
classification.config  reference.config      snort.debian.conf     unicode.map
community-sid-msg.map  rules                 snort.lua
```

Configuration file (-c) can be started like this:

snort -c /etc/snort/snort.lua

In configuration file we can specify which rule files to enable, which network range to monitor and allow other settings. The rule files are stored in the rules folder.

```
ubuntu@tryhackme:/etc/snort$ ls rules
attack-responses.rules      community-web-dos.rules    policy.rules
backdoor.rules              community-web-iis.rules    pop2.rules
bad-traffic.rules           community-web-misc.rules   pop3.rules
chat.rules                  community-web-php.rules    porn.rules
community-bot.rules         ddos.rules                 rpc.rules
community-deleted.rules     deleted.rules              rservices.rules
community-dos.rules         dns.rules                  scan.rules
community-exploit.rules     dos.rules                  shellcode.rules
community-ftp.rules          experimental.rules         smtp.rules
community-game.rules        exploit.rules              snmp.rules
community-icmp.rules        finger.rules               sql.rules
community-imap.rules        ftp.rules                  telnet.rules
community-inappropriate.rules icmp-info.rules            tftp.rules
community-mail-client.rules icmp.rules                  virus.rules
community-misc.rules         imap.rules                 web-attacks.rules
community-nntp.rules         info.rules                 web-cgi.rules
community-oracle.rules       local.rules                web-client.rules
community-policy.rules       misc.rules                 web-coldfusion.rules
community-sip.rules          multimedia.rules           web-frontpage.rules
community-smtp.rules         mysql.rules                web-iis.rules
```

Rule format

alert icmp any any -> \$HOME_NET any (msg:"Ping Detected"; sid: 10001; rev:1;)

Alert = action	This specifies which action to take when the rule triggers. In our case this is alert action.
Icmp = protocol	In this case we use ICMP protocol when pinging a host
any any (for any detection)	Source IP and source port

\$HOME_NET any	Destination IP and destination port
msg:"Ping Detected"	Message displayed when the rule is triggered
Signature ID = sid	A unique identifier of the rule. Each rule has its own unique identifier.
Rule revision = rev	Every time the rule is modified, its revision number is incremented, which helps in tracking changes to any rule.

Rule creation

We will paste the rule from above to our local.rules file.

```

ubuntu@tryhackme: /etc/snort
File Edit View Search Terminal Help
GNU nano 4.8 /etc/snort/rules/local.rules Modified
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert icmp any any -> $HOME_NET any (msg:"Ping Detected"; sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"SSH Connection Detected"; sid:1000002; rev:1;)
alert icmp any any -> 127.0.0.1 any (msg:"Loopback Ping Detected"; sid:10003; rev:1;)
  
```

Then we press CTRL+X and then y for saving the changes.

Rule testing

At first we start the Snort tools to detect any intrusions defined in the rule file.

```

ubuntu@tryhackme: /etc/snort$ sudo snort -q -l /var/log/snort -i lo -A alert_fast
-c /etc/snort/snort.lua
  
```

We opened different tab and ping 127.0.0.1 and boom we have gotten the detection.

```
06/17-14:22:44.355374 [**] [1:10003:1] "Loopback Ping Detected" [**] [Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1
06/17-14:22:44.355388 [**] [1:1000001:1] "Ping Detected" [**] [Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1
06/17-14:22:44.355388 [**] [1:10003:1] "Loopback Ping Detected" [**] [Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1
06/17-14:22:45.379354 [**] [1:1000001:1] "Ping Detected" [**] [Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1
06/17-14:22:45.379354 [**] [1:10003:1] "Loopback Ping Detected" [**] [Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1
06/17-14:22:45.379366 [**] [1:1000001:1] "Ping Detected" [**] [Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1
```

If we want to run our local.rules we need to run configuration file, which basically runs the rules for us!!

Running Snort on PCAP files

We saw how Snort can be used for IDS on real-time traffic. However, you may sometimes encounter a scenario where you have historical network traffic logged in a file, and you have to perform a forensic investigation to determine any signs of intrusion through that traffic. This traffic is usually logged in the standard packet capture format “PCAP”. Snort is also equipped to perform detections on these PCAP files containing historical network traffic.

This is the command that would help us:

sudo snort -q -l /var/log/snort -r Task.pcap -A alert_fast -c /etc/snort/snort.lua

PRACTICAL

We are a third-party forensic investigator. A company contacts us to investigate a recent attack on their network. They handed over a PCAP file named "Intro_to_IDS.pcap", which contains the network traffic captured during the attack. Your task is to run Snort on this PCAP file and answer the questions given in this task.

So we will use the the command from above.

```
sudo snort -q -l /var/log/snort -r Intro_to_IDS.pcap -A alert_fast -c /etc/snort/snort.lua
```

```
ubuntu@ubuntu:~/etc/snort$ sudo snort -q -l /var/log/snort -r Intro_to_IDS.pcap -A alert_fast -c /etc/snort/snort.lua
07/18-12:52:59.337559 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.510662 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.510663 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.693619 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.696547 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.867597 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.868435 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.050645 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.057313 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.272189 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.444488 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.623967 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.647745 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.830895 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.830918 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.364435 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.366033 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.536660 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.710516 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.710517 ** [1:1000002:1] "SSH Connection Detected" ** [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
```

- 1) What is the IP address of the machine that tried to connect to the subject machine using SSH?

10.11.90.211

- 2) What other rule message besides the SSH message is detected in the PCAP file?

```
07/18-12:53:16.954377 ** [1:1000001:1] "Ping Detected" ** [Priority: 0] (ICMP) 10.10.161.151 -> 10.11.90.211
07/18-12:53:17.956812 ** [1:1000001:1] "Ping Detected" ** [Priority: 0] (ICMP) 10.11.90.211 -> 10.10.161.151
07/18-12:53:17.956842 ** [1:1000001:1] "Ping Detected" ** [Priority: 0] (ICMP) 10.10.161.151 -> 10.11.90.211
07/18-12:53:18.972925 ** [1:1000001:1] "Ping Detected" ** [Priority: 0] (ICMP) 10.11.90.211 -> 10.10.161.151
07/18-12:53:18.972956 ** [1:1000001:1] "Ping Detected" ** [Priority: 0] (ICMP) 10.10.161.151 -> 10.11.90.211
```

ICMP rule. > Ping Detected

- 3) What is the sid of the rule that detects SSH?

1000002

This task was simple, but if there is a bigger file than just a few logs we will use grep command for better orientation and getting the unique IP addresses.

For example:

```
sudo tshark -r capture.pcap > log.txt
```

and from the txt file → **grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' log.txt | sort -u**